Lesson Activities — 1.2 Software & Software Development

Teacher-facing pack of in-lesson activities for OCR H446 Section 1.2 — varied, mostly zero-prep, built for talk, movement and deep processing rather than exam drilling.

1.2.1 Operating Systems

The Interrupted Classroom (interrupt role-play · 12 min · whole class)

Resources: A bell (or phone alarm); a passage for students to copy (any paragraph on the board); sticky notes or the corner of the page as the "stack". **How it runs:** 1. Without explaining why, set students copying the passage neatly — this is their "current process". Tell them accuracy matters and they'll resume later. 2. Ring the bell mid-sentence. Students must immediately write their exact position (word number and letter) on a sticky note — "push your registers onto the stack" — then service the interrupt: a quick unrelated task on the board (e.g. "write today's date in binary"). 3. Ring a *higher-priority* bell (double ring) during the first task: they must push a second sticky note recording their position in the binary task, service the new task (stand up, push chair in, sit down), then pop back — finish the binary, then finish the sentence, in exactly that order. 4. Debrief: map every action to the theory — bell = interrupt signal; sticky note = pushing PC/ACC onto the stack; the board task = ISR; double bell = higher-priority interrupt nesting; resuming from the note = popping the stack. 5. Ask the trap question: "When was I *allowed* to ring the bell, in CPU terms?" (The CPU only checks the interrupt register at the end of each F–D–E cycle — it finishes the current instruction first.) **Answers / expected outcomes:** Students articulate the full sequence: check interrupt register at end of cycle → compare priority → push registers (PC, ACC) onto the stack → load the ISR address → run ISR → pop registers → resume. Nesting works because the stack is LIFO — the second sticky note comes off first. **Stretch:** "What if interrupts arrived faster than the ISRs complete?" — link to why interrupts have priorities and to the polling alternative interrupts replace.

The Photocopier Queue: Human Scheduling (kinesthetic simulation · 15 min · 5 volunteers + whole class)

Resources: Job cards for four "processes": A = 4 units, B = 1, C = 2, D = 3 (units = squats, star jumps, or tally marks at the board). One student is the CPU; class keeps a wait-time tally per process on the board. **How it runs:** 1. Round 1 — FCFS: processes queue in order A, B, C, D; CPU runs each to completion. Class records each process's waiting time (time before first/remaining service is complete: use turnaround − burst). 2. Round 2 — Round robin, quantum 1: unfinished processes rejoin the back of the queue. Track it visibly; let the class call out queue order. 3. Round 3 — SJF (non-pre-emptive): CPU picks the shortest total job next. 4. Compare the three averages on the board; ask B (the 1-unit job) which scheduler it prefers, and A which one it fears (starvation discussion). 5. Finish: "Which is pre-emptive?" and "What does RR need that FCFS doesn't?" (a timer interrupt — link back to the bell activity). **Answers / expected outcomes:** All arrive at t=0, order A,B,C,D. FCFS completes A=4, B=5, C=7, D=10 → waits 0,4,5,7 (avg 4.0). SJF runs B,C,D,A → waits 0,1,3,6 (avg 2.5). RR q=1 completes B=2, C=6, D=9, A=10 → waits (turnaround−burst) A 6, B 1, C 4, D 6 (avg 4.25) but no long wait before *first* service — fairness vs efficiency trade-off. Pre-emptive: RR (and SRT/MLFQ); FCFS and SJF are not. **Stretch:** Re-run with D arriving at t=2 under SRT and have the class detect the pre-emption point.

Always–Sometimes–Never: The OS Edition (statement judgement · 10 min · pairs, mini-whiteboards)

Resources: Statement list on the board; whiteboards showing A / S / N. **How it runs:** 1. Reveal statements one at a time; pairs commit A/S/N on whiteboards, then defend on request. 2. Statements: (a) "Virtual memory increases the amount of RAM in the computer." (b) "Paging divides memory into fixed-size blocks." (c) "A device driver works with any operating system." (d) "The BIOS is the first program to run when the computer starts." (e) "An embedded OS is stored in ROM." (f) "A real-time OS is faster than a desktop OS." (g) "A pre-emptive scheduler can stop a running process." 3. For every "Sometimes", the pair must state the condition; for every "Never", the correct version. 4. Pairs write one new S statement about virtual machines and test it on a neighbouring pair. **Answers / expected outcomes:** (a) Never — it uses secondary storage *as if* it were RAM, and it's slower. (b) Always — that's the paging/segmentation discriminator. (c) Never — drivers are OS- and hardware-specific. (d) Always — then POST, then bootstrap loads the OS. (e) Sometimes/usually — typical but not definitional; reward the nuance. (f) Never as stated — it *guarantees a response within a deadline*, which is not the same as "faster". (g) Always — that is the definition of pre-emptive. **Stretch:** Convert (f) into an Always statement that would earn full definition marks.

Which OS Am I? Four Corners (kinesthetic sort · 8 min · whole class)

Resources: Four corners labelled DISTRIBUTED / EMBEDDED / MULTI-USER / REAL-TIME; centre of the room = MULTITASKING DESKTOP. **How it runs:** 1. Read a system; students move within five seconds: pacemaker; washing machine; a university mainframe with 400 logged-in students; Google's search infrastructure across thousands of servers; a Windows laptop; a car's anti-lock braking system; a smart TV; a cloud server farm renting compute to many companies. 2. Interview split votes — several systems legitimately claim two corners (a pacemaker is embedded *and* real-time). 3. For dual-claim systems, the two corners must each state the *defining* feature they're claiming (deadline guarantee vs built-into-one-device). 4. Finish on whiteboards: one-sentence definition of the corner they visited most. **Answers / expected outcomes:** Pacemaker/ABS: real-time (embedded also credited — the discriminator is the guaranteed response deadline). Washing machine/smart TV: embedded. Mainframe: multi-user. Google/server farm: distributed (farm renting to companies also multi-user — reward justified moves). Laptop: multitasking. Success = students quoting defining features, not examples. **Stretch:** Name a single real system that defensibly belongs to three corners at once and argue it (e.g. an aircraft's avionics network).

The OS Concept Web (concept mapping · 12 min · pairs)

Resources: Plain A4/A3 per pair; the word bank on the board: OS, abstraction, memory management, paging, segmentation, virtual memory, page fault, thrashing, interrupt, ISR, stack, scheduler, round robin, pre-emptive, BIOS, POST, driver, virtual machine, intermediate code. **How it runs:** 1. Pairs place "OS" centrally and connect terms with labelled arrows — the rule: every arrow must carry a verb phrase ("*swaps pages using*", "*is loaded by*"), because unlabelled lines score nothing. 2. Set two compulsory bridges: connect *interrupt* to *scheduler*, and *virtual memory* to *thrashing*, each via a labelled path. 3. Gallery walk: pairs leave their map, tour two others, and add one sticky-note challenge ("prove this link") to each. 4. Return and defend or fix challenged links. **Answers / expected outcomes:** Valid bridges: timer *interrupt* → triggers → *scheduler* → pre-empts process (round robin); *virtual memory* → excessive swapping/page faults → *thrashing*. Quality criteria: BIOS → runs POST → loads OS (not "BIOS is the OS"); driver → lets OS control → specific device; VM → runs → intermediate code. Misconception flags: cache or RAM arrows into "virtual memory adds RAM". **Stretch:** Add three 1.1 terms (cache, FDE cycle, registers) and integrate them legally into the web.

1.2.2 Applications Generation

Compilation Assembly Line (human sort · 12 min · 10 volunteers + class as inspectors)

Resources: 10 cards, one per volunteer: "source code arrives"; "comments and whitespace removed"; "code converted into tokens"; "symbol table created"; "tokens checked against grammar rules"; "parse tree built"; "syntax errors reported"; "object/machine code produced"; "code refined to run faster / use less memory"; "same output, better performance". **How it runs:** 1. Shuffle and deal the cards; volunteers arrange themselves in one line at the front. The seated class are inspectors who may shout "STOP THE LINE" and state a fault, factory-style. 2. Once the line settles, the teacher walks it: each student reads their card and names the *stage* they belong to. 3. Class groups the line into the four named stages by physically inserting four students holding stage-name signs: LEXICAL ANALYSIS / SYNTAX ANALYSIS / CODE GENERATION / OPTIMISATION. 4. Sabotage round: teacher secretly swaps two students during a "close your eyes" moment; inspectors must find and fix the swap. 5. Exit question to the line: "Which stage reports a missing bracket, and why can't lexical analysis catch it?" **Answers / expected outcomes:** Order: Lexical (remove comments/whitespace → tokens → symbol table) → Syntax (check vs grammar → parse tree → syntax errors reported) → Code generation (object/machine code) → Optimisation (faster/less memory, same result). Missing bracket = syntax analysis, because lexical analysis only tokenises — it has no grammar to check against. **Stretch:** Add two rogue cards — "program is executed" and "variables are given values" — which belong nowhere in compilation and must be ejected with reasons.

The Great Software Debate: Open the Source? (structured debate with role cards · 20 min · groups of 6)

Resources: Motion on the board: "This school should move entirely to open source software." Six role cards per group (write on board, no printing needed): School network manager (worried about support and staff training) · Startup CTO (built a business on Linux) · Hospital IT lead (safety-critical, needs accountability) · Indie game developer (sells closed-source games to eat) · Security researcher (wants to audit code) · Software vendor sales rep (sells proprietary licences). **How it runs:** 1. Groups of six; each student draws a role and gets 3 minutes with the knowledge organiser to build two arguments *in character* — they must argue the role's interest, not their own opinion. 2. Round-table: 45 seconds each, no interruptions; then 4 minutes of open cross-examination in character. 3. Roles vote on the motion *as their character would*, then step out of role and vote again as themselves; groups compare the two tallies. 4. Whole class: harvest the strongest argument heard on each side; teacher boards them under Open / Closed. 5. Close with the precision check: "Open source means free of charge — true?" **Answers / expected outcomes:** Strong open-source arguments: source available to inspect/modify, usually free, community support, security through auditability, no vendor lock-in. Strong closed-source: official paid support and accountability, polished/consistent products, revenue funds development, code secrecy protects business model. Precision check: false — open source concerns *source availability and modification rights*; it is *usually* free but that is not the definition. The two-vote comparison shows how context (role) drives the "right" answer — exactly what scenario questions want. **Stretch:** Security researcher and vendor rep swap cards and must steelman their former opponent's best point.

Spot and Fix: Compiler vs Interpreter Horror Story (error hunt · 10 min · pairs)

Resources: Projected paragraph (below); mini-whiteboards. **How it runs:** 1. Project: *"A compiler translates a high-level program one line at a time while it runs, producing a standalone executable. An*

interpreter translates the whole program in one go before it runs, so interpreted programs run faster afterwards. A compiled .exe needs the source code present to run, which is why companies distribute their source with the executable. Interpreters report every error in the program in one list at the end, whereas compilers stop at the first error. An assembler translates high-level code into assembly language." 2. Pairs find the errors (tell them: 7) and number them. 3. Pairs rewrite the paragraph correctly in exactly four sentences. 4. Compare rewrites; class votes for the tightest correct version. **Answers / expected outcomes:** (1) Compiler translates *all at once, before* running, not line-by-line; (2) it's the compiler that produces the executable — statement half-right but attached to wrong behaviour; (3) interpreter translates *line by line at run time*; (4) *compiled* programs run faster; (5) a compiled .exe does NOT need source (that's why companies can keep source secret); (6) errors swapped: compilers report all at the end of translation, interpreters stop at the first error hit; (7) an assembler translates *assembly language into machine code*, roughly one-to-one. **Stretch:** Add the portability comparison the paragraph never mentions, in one accurate sentence.

Utility or Application? Stand Up / Sit Down (zero-prep sort · 5 min · whole class)

Resources: None. **How it runs:** 1. Stand = utility, sit = application. Call fast: word processor; antivirus; disk defragmenter; web browser; file compression tool; spreadsheet; automatic backup tool; video game; firewall; photo editor. 2. On each split decision, freeze and take one advocate from each posture. 3. Ask the class for the *rule*, not the list: what test decides it? 4. Sting in the tail: "Windows — stand or sit?" (Neither — it's an OS/system software; catches list-memorisers.) **Answers / expected outcomes:** Utilities (stand): antivirus, defrag, compression, backup, firewall — software that maintains/optimises the computer itself. Applications (sit): word processor, browser, spreadsheet, game, photo editor — software for a real-world user task. The classic trap is antivirus = utility, not application. Rule: "does it serve the user's task or the machine's upkeep?" **Stretch:** Name a program that defensibly sits in both camps and argue it (e.g. a file manager or cloud-sync client).

Build Me an Executable (mini role-play · 10 min · 6 volunteers + class)

Resources: Cards: "your compiled code", two "library routines (pre-compiled, tested)", "LINKER", "LOADER", a chair labelled "RAM", a corner labelled "storage". **How it runs:** 1. Three students (your code + two library routines) stand scattered in "storage". Your code holds gaps: unresolved calls written as "??? sqrt" and "??? print". 2. LINKER physically gathers the three, resolves the gaps by writing the routine-holders' names over the ???s, and ties them together (linked arms) into one executable. Ask: "Static or dynamic linking?" (Static — code copied in; the blob is bigger but self-contained.) 3. Re-run as dynamic: LINKER writes only *addresses* on sticky notes instead of joining arms; the libraries stay put in storage until run time and could be shared by another program (second "program" volunteer points at the same libraries). 4. LOADER escorts the executable from "storage" to the RAM chair to run. 5. Class states in one sentence each what linker, loader and library mean. **Answers / expected outcomes:** Library = pre-written, pre-compiled, tested routines reused to save time and improve reliability. Linker = combines compiled code with libraries/modules, resolving references — static copies code in (larger, self-contained), dynamic links at run time (smaller, shared, updatable without recompiling). Loader = OS software that copies the executable into RAM ready to run. **Stretch:** "The dynamic library gets updated and the update is buggy — what happens to every program that links it, and why can't that happen with static linking?"

1.2.3 Software Development

Methodology Expert Jigsaw (jigsaw peer-teaching · 20 min · home groups of 5)

Resources: Knowledge organiser methodology table; A5 paper per student. **How it runs:** 1. Home groups of five number off: (1) Waterfall, (2) Agile, (3) XP, (4) Spiral, (5) RAD. Experts regroup by number. 2. Expert groups agree: the key idea, one strength, one weakness, and the *tell-tale scenario clue* that screams "pick me" (e.g. Spiral: "large, expensive, risky"). 3. Back in home groups, experts teach for 60 seconds each; listeners' pens down until each minute ends. 4. Teacher reads three scenarios; home groups hold up the methodology and the *clue-word* from the scenario that justified it: (a) "a bank needs a payroll system with legally fixed requirements"; (b) "a startup wants an app but keeps changing its mind, needs something usable next month"; (c) "a space agency's crewed-flight software, huge budget, failure is catastrophic". 5. Individually, books closed: everyone writes the strength/weakness of a methodology they did *not* teach. **Answers / expected outcomes:** (a) Waterfall — fixed, clear requirements. (b) Agile or RAD — changing requirements/speed; accept either with the clue named (RAD's clue: prototype fast; Agile's: iterative feedback). (c) Spiral — risk analysis each loop on a large, high-risk project. Expert sheets must match the organiser (XP = pair programming, constant testing, customer on team). **Stretch:** Experts must also say what their methodology *costs* you (Waterfall: client sees nothing till the end; Agile: unpredictable cost/time; XP: staff-intensive; Spiral: expensive/complex; RAD: robustness, needs committed users).

Waterfall ↔ Agile Decision Continuum (physical continuum · 10 min · whole class)

Resources: Line across the room: one wall "pure Waterfall", other "pure Agile". Scenario list. **How it runs:** 1. Read a scenario; students take a position on the line and must be able to name the scenario clue that put them there. 2. Scenarios: (a) replacing an air-traffic-control system where regulators demand full documentation up front; (b) two founders building a social app for a market that shifts monthly; (c) a school website for a client who "will know what they want when they see it"; (d) a fixed-price contract with penalty clauses for missed spec items; (e) a game studio prototyping ideas for an unannounced console. 3. Interview extremes and the middle; middle-standers must say what *hybrid* they'd actually run. 4. Swap: three students must move to the opposite half of the line and steelman that position for the same scenario. **Answers / expected outcomes:** (a), (d) toward Waterfall — fixed, documented, contractual requirements. (b), (c), (e) toward Agile/RAD — vague or volatile requirements, need feedback/prototypes. Success = clue-to-choice reasoning ("penalty clauses ⇒ fixed spec ⇒ Waterfall"), not brand loyalty. Middle answers should describe iterative development within documented milestones. **Stretch:** "Where on this line does Spiral live, and why is that a trick question?" (It's iterative like Agile but heavyweight and documentation-rich like Waterfall — risk analysis is the axis the line doesn't show.)

Just a Minute: The Lifecycle Gauntlet (speaking challenge · 10 min · groups of 4)

Resources: Topic slips: analysis, design, development, testing, evaluation, plus wildcard "the whole lifecycle". **How it runs:** 1. Groups of four: speaker, timer, two challengers. 60 seconds on the drawn stage without hesitation, repetition of the stage name, or technical error. 2. Challengers buzz to steal the floor; whoever is speaking at 0:00 scores. 3. House rule for this topic: the speaker must include at least one *named output* of their stage (e.g. design → data structures/algorithms/interface designs) or the point is void. 4. Teacher adjudicates disputed claims at the end, correcting the record aloud. **Answers / expected outcomes:** Named outputs — analysis: requirements spec + feasibility; design: data structures, algorithms, interface, I/O formats; development: modular code; testing: results against normal/boundary/erroneous data, alpha/beta findings; evaluation: judgement vs original requirements (effective, usable, robust, maintainable). Common disputed claim to correct: "testing only happens after

development" (iterative methodologies test throughout). **Stretch:** Wildcard speakers must traverse all five stages in order within the minute.

Doomed by Design (think-pair-share · 10 min · pairs)

Resources: None — prompt on the board: "Design the *worst possible* methodology choice." **How it runs:** 1. Think: each student picks a project scenario and secretly assigns it the most catastrophically wrong methodology (e.g. Waterfall for a startup that pivots weekly; XP with one solo developer; RAD for a pacemaker). 2. Pair: swap disasters; the partner must narrate *specifically how* the project dies (which stage, which artefact missing, which stakeholder rages) using correct terminology. 3. Share: funniest-but-technically-accurate disaster is performed for the class in 30 seconds. 4. Flip: the class prescribes the correct methodology and names the scenario clue that dictates it. **Answers / expected outcomes:** Failure narratives must invoke real trade-offs: Waterfall dies when requirements change after sign-off and the client sees nothing until the end; XP needs pairs and an on-site customer; RAD's rapid prototypes are the wrong foundation for safety-critical robustness (a pacemaker needs Spiral/heavy documentation and risk analysis). Prescriptions must cite clues (volatility → Agile; risk+scale → Spiral; speed+vagueness → RAD; fixed spec → Waterfall). **Stretch:** Ruin a project *twice*: show how even the right methodology fails if one of its preconditions (committed users, skilled devs, available customer) is missing.

1.2.4 Types of Programming Language

The Human LMC (unplugged role-play · 15 min · groups of 6 or whole class)

Resources: Desks as numbered mailboxes; instruction cards laid at mailboxes 0–4; a student as the Little Man (fetch/decode/execute), one as ACC (holds a whiteboard with the current value), one as PC, class as verifiers. Program: `00 INP · 01 LOOP OUT · 02 SUB ONE · 03 BRP LOOP · 04 HLT · 05 ONE DAT 1`. **How it runs:** 1. Set the room: cards at mailbox positions; PC holds "00"; teacher plays the in/out tray. Input value: 3. 2. The Little Man performs each cycle aloud: walk to the mailbox PC names, read the card, PC increments, then execute — updating ACC's whiteboard or shouting outputs to the tray. 3. Class verifiers log every output and every ACC value; anyone spotting an illegal move (e.g. Little Man changing PC without a branch) stops play. 4. At `BRP`, the Little Man must ask ACC "are you zero or positive?" before overwriting PC's card — the class predicts the answer first. 5. Run to `HLT`; then ask: "Why did 0 get output but −1 didn't?" and "What single card would you change to stop at 1 instead of 0?" **Answers / expected outcomes:** With input 3, outputs are **3, 2, 1, 0**: ACC goes $3 \rightarrow \text{out} \rightarrow 2 \rightarrow \text{out} \rightarrow 1 \rightarrow \text{out} \rightarrow 0 \rightarrow \text{out} \rightarrow -1$ (BRP fails, falls through to HLT). BRP branches while ACC ≥ 0, so 0 is printed before the loop exits. To stop after 1: swap `BRP` for `BRZ`-based logic or reorder to subtract before output (accept any correct fix, e.g. move `SUB ONE` before `OUT` and adjust — reward traced justification). **Stretch:** Rewrite the program using indirect addressing to walk through and output a list of DAT values (introduce a pointer mailbox).

Addressing Modes Four Corners (kinesthetic sort · 10 min · whole class)

Resources: Corners labelled IMMEDIATE / DIRECT / INDIRECT / INDEXED. Memory table projected: address 20 → 35, address 35 → 99, index register = 3, address 23 → 7. **How it runs:** 1. Baseline: "The instruction is `LDA 20`. I want the value **20** in the accumulator — go to the mode that does that." Repeat for wanting **35**, then **99**, then **7**. 2. Any student in a minority corner argues their case before the reveal; the majority must produce the rule, not just the answer. 3. Flip the game: teacher stands in a corner; class writes on whiteboards what `LDA 20` loads from there. 4. Application round: "You're implementing an array traversal — which corner? A pointer — which corner?" Students move and justify. **Answers / expected**

outcomes: Value 20 = immediate (operand *is* the value); 35 = direct (operand is the address holding the value); 99 = indirect (address 20 holds address 35, whose content 99 is loaded); 7 = indexed (base 20 + index 3 = address 23 → 7). Arrays → indexed (increment index register); pointers → indirect. **Stretch:** With the same table, invent a memory state where direct and indirect load the *same* value, and explain what that says about address 20 (it must hold an address that stores itself, e.g. 20→20).

OOP Concept Web With Compulsory Bridges (concept mapping · 12 min · pairs)

Resources: A4/A3 per pair; word bank: class, object, instantiation, attribute, method, encapsulation, inheritance, polymorphism, superclass, subclass, private, public. **How it runs:** 1. Pairs build a concept map where every arrow carries a verb phrase ("*is an instance of*", "*hides data using*"). 2. Compulsory bridges: connect *encapsulation* to *private*; connect *polymorphism* to *inheritance*; connect *object* to *class* in both directions with different verbs. 3. Trade maps with another pair, who must add one true link the authors missed, in another colour. 4. Authors approve or veto the addition — vetoes must cite a definition. **Answers / expected outcomes:** Valid bridges: encapsulation → makes attributes → private → accessed via public methods; polymorphism → typically achieved by subclasses *overriding* methods → inherited from a superclass; object *is an instance of* class / class *is the blueprint for* object; instantiation → creates → object. Red-flag links to catch: "inheritance = copying code" (it's acquiring, with add/override), attributes as subroutines (attributes are variables; methods are the subroutines). **Stretch:** Integrate a concrete mini-scenario (Vehicle → Car/Bike with `move()`) into the map so every abstract term has a labelled example.

Analogy Autopsy: Cookie Cutters and Blueprints (analogy critique · 8 min · think-pair-share)

Resources: Board: "A class is a cookie cutter; objects are the cookies." **How it runs:** 1. Think: list what the analogy captures correctly (1 min). 2. Pair: find at least two breakdown points (2 min). 3. Share and board under Holds up / Breaks down; then repeat quickly for a second analogy: "Inheritance is like inheriting money from a parent." 4. Pairs draft a one-sentence replacement analogy for the weakest one and read them out; class picks the best. **Answers / expected outcomes:** Cookie cutter — holds up: one template, many instances, same shape/structure. Breaks down: cookies can't *do* anything (objects have methods/behaviour); all cookies are identical, but objects hold different attribute *values* (state); nothing represents encapsulation or the class's own definition changing. Money inheritance — breaks down badly: the parent must *die/lose* the money, whereas a superclass keeps everything and many subclasses inherit simultaneously; subclasses can also *override* and *extend*, heirs can't rewrite what they inherit. **Stretch:** Write an analogy for polymorphism (e.g. "press *play* on different devices") and submit it for autopsy by another pair.

Whiteboard Quick-Draw: Class Diagram From Spec (drawing from memory · 10 min · individual, mini-whiteboards)

Resources: Mini-whiteboards; spec read aloud twice, not shown: "A `Vehicle` class has a private attribute `speed` and public methods `accelerate()` and `getSpeed()`. `ElectricCar` inherits from `Vehicle`, adds a private attribute `charge`, and overrides `accelerate()`." **How it runs:** 1. Read the spec twice; students draw the full class diagram from the spoken spec only. 2. Show boards; peer-mark against criteria projected afterwards. 3. Targeted fix: everyone checks the direction of their inheritance arrow specifically, and the +/− symbols. 4. Redraw in 90 seconds from memory, then compare with attempt one. **Answers / expected outcomes:** Two boxes each split into name / attributes / methods; `Vehicle`: `- speed`, `+ accelerate()`, `+ getSpeed()`; `ElectricCar`: `- charge`, `+ accelerate()` (listed again = override). Inheritance arrow points **from child to parent** (ElectricCar → Vehicle) — the single most-lost mark; `-` = private, `+` = public. Speed appears only in Vehicle (inherited, not redrawn). **Stretch:** Add a `Bicycle`

subclass that would make `accelerate()` polymorphic across three classes, and annotate where encapsulation forces `getSpeed()` to exist at all.